

Credit Risk Intelligence Platform

An AI-powered platform for loan default prediction, explainability, and natural-language data access



Building a bank-ready credit risk platform

Multi-table data

Joined application, bureau, prior-loan, and monthly repayment history — not just a single table

Explainable by design

Every prediction is backed by SHAP reasoning and auditable business rules, not a black box

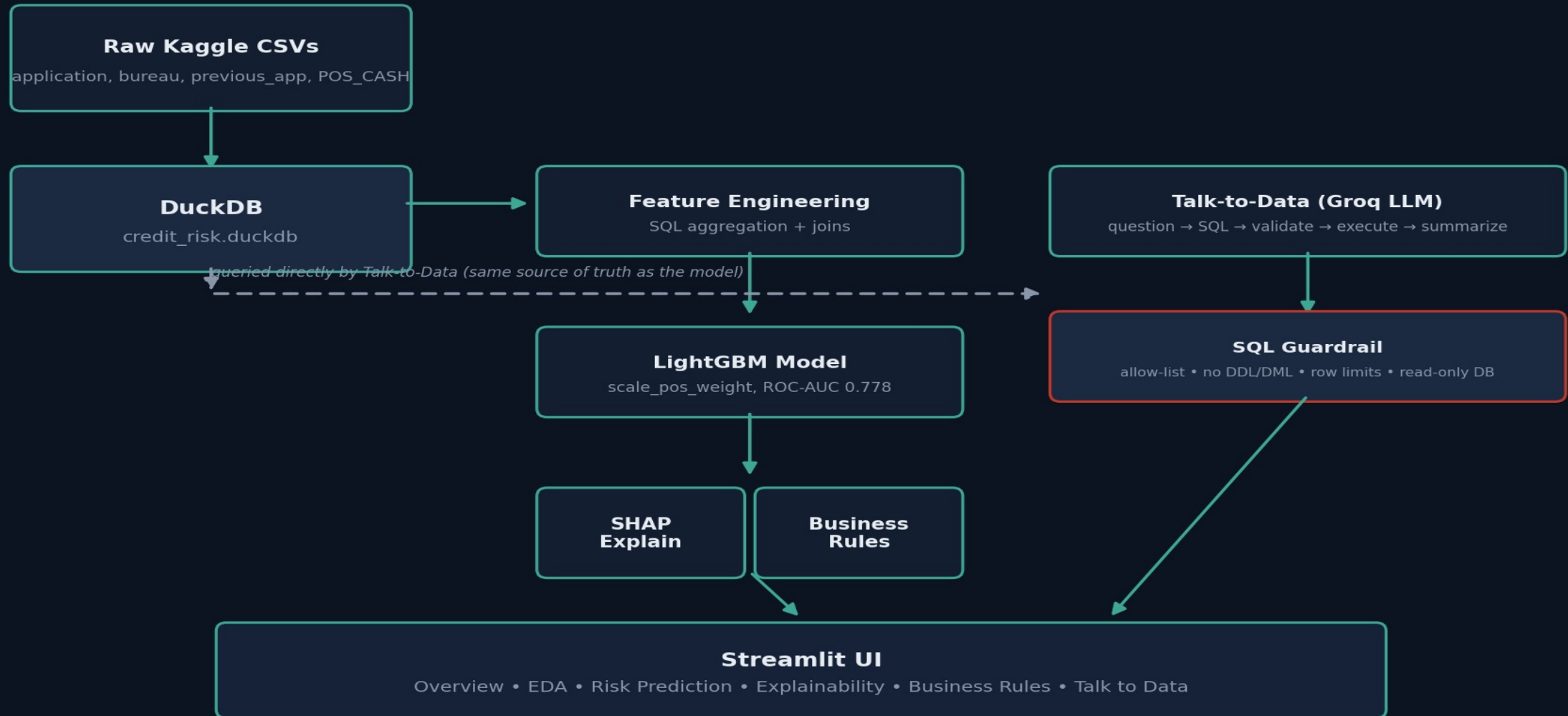
Talk to the data

A guardrailed NL-to-SQL chatbot for non-technical stakeholders to explore the data directly

Ship it

Fully Dockerized, single-command deployment

How the pieces fit together



Multi-table feature engineering, not just one CSV

Most submissions on this dataset use application_train.csv alone. This platform joins three additional tables — aggregated in SQL for speed on 10M+ row tables — to capture signal a single-table model would miss.

applications

307,511 rows

Core applicant demographics & loan terms

bureau

1.7M rows

External credit bureau history — active/overdue loans, debt ratios

previous_applications

1.7M rows

This applicant's prior loans with the same lender

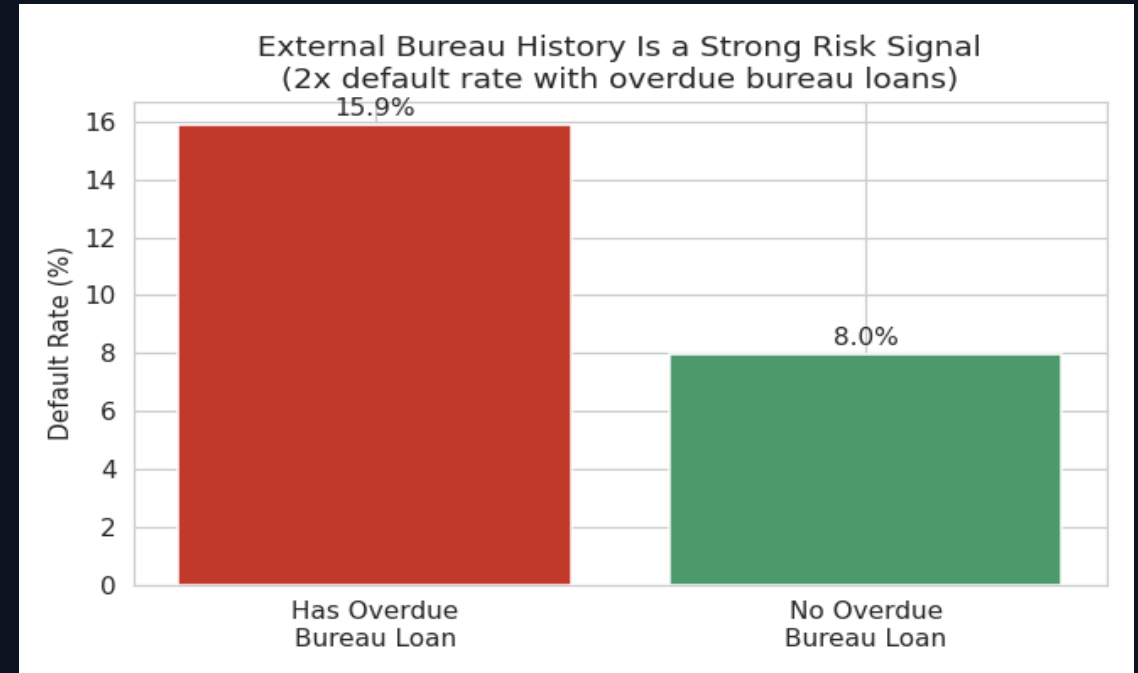
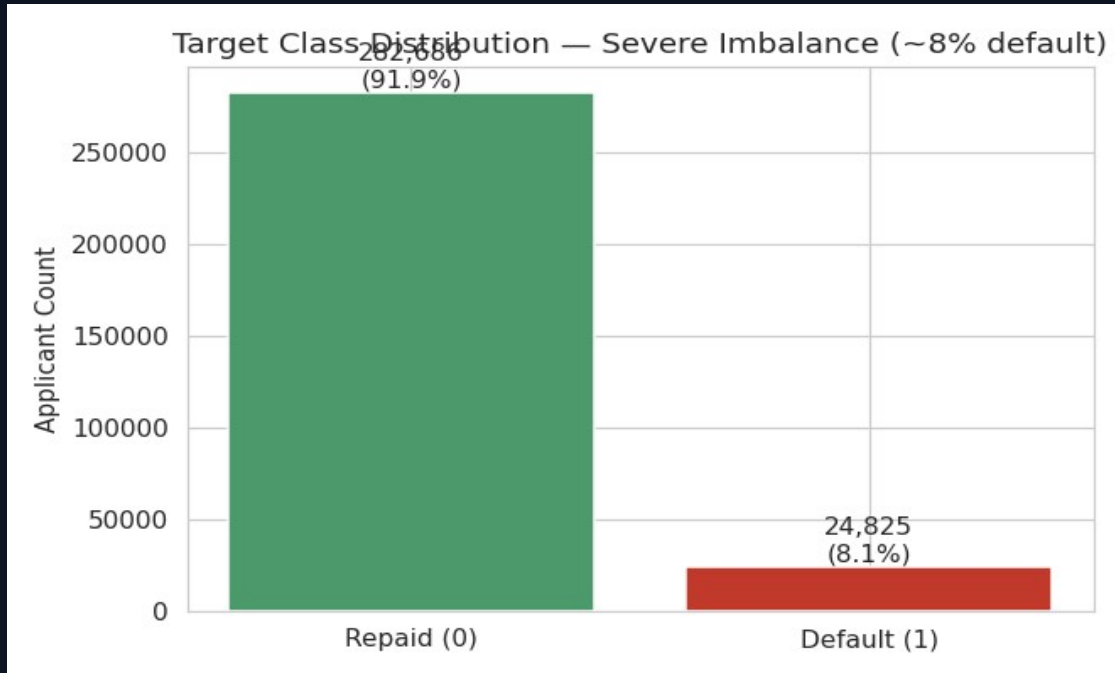
pos_cash_balance

10M rows

Monthly days-past-due (DPD) — direct repayment behavior signal

Result: 152 features across 4 tables, feeding a single unified model.

What the data says before any model is built



- Only 8.07% of applicants default — a naive "approve everyone" model would be 92% "accurate" and useless
- An overdue external bureau loan nearly doubles default risk (15.9% vs 8.0%)

LightGBM with an evidence-based imbalance strategy

ROC-AUC

0.778

PR-AUC

0.273

Recall @ 0.5

66.2%

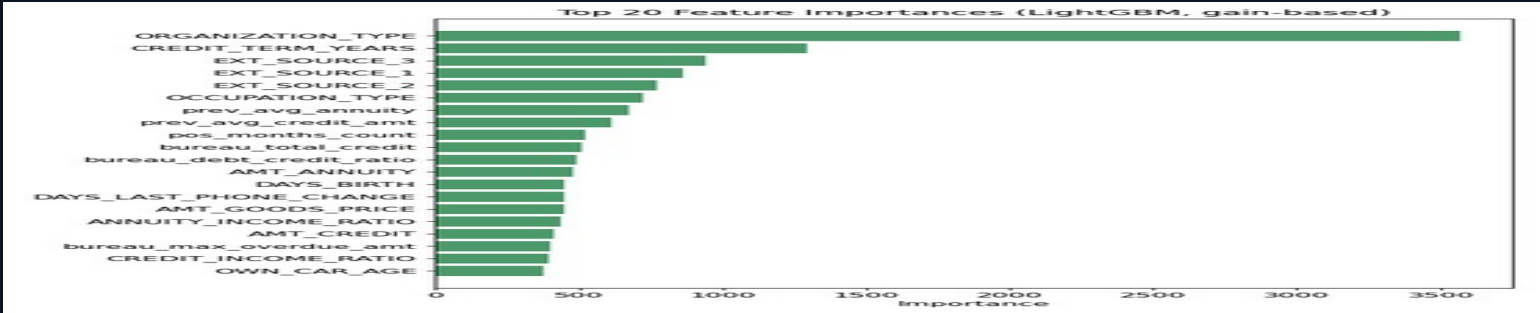
Features

152

Class imbalance (~8% default rate): scale_pos_weight vs SMOTE — tested, not assumed

Approach	Validation ROC-AUC	Notes
scale_pos_weight (production)	0.7782	Simpler, faster, no synthetic data
SMOTE (0.5 ratio)	0.7793	Statistically indistinguishable (difference = 0.001)

Evaluated on ROC-AUC / PR-AUC, not accuracy — accuracy is misleading under 8% class imbalance.



SHAP: auditable reasons behind every prediction

- TreeExplainer — exact SHAP values for gradient-boosted trees, not an approximation
- Per-applicant: top features, direction, and a plain-English summary
- Global: beeswarm plot across a 2,000-applicant sample
- Example: correctly flagged 22 active bureau loans + 70.7% refusal rate on a genuinely-defaulted applicant (p=0.955)

«

◆ Credit Risk Platform

AI-powered credit risk intelligence — NeoStats assignment

Overview

Data Exploration (EDA)

Risk Prediction

Explainability

Business Rules

Talk to Data

Deploy

Explainable AI

SHAP-based explanation of individual predictions — auditable, feature-level reasoning.

High Risk

Default probability: 35.8%

Summary: Risk was pushed UP mainly by: amt goods price, organization type, credit term years. Risk was pulled DOWN mainly by: own car age, ext source 1, bureau debt credit ratio.

Top contributing factors:

▼

OWN_CAR_AGE = 9.0 (decreased risk, impact 0.180)

▲

AMT_GOODS_PRICE = 450000.0 (increased risk, impact 0.178)

▼

EXT_SOURCE_1 = 0.5 (decreased risk, impact 0.128)

▲

ORGANIZATION_TYPE = Business Entity Type 3 (increased risk, impact 0.124)

▼

bureau_debt_credit_ratio = 0.22343412178294153 (decreased risk, impact 0.121)

▲

CREDIT_TERM_YEARS = 1.6666666666666667 (increased risk, impact 0.114)

▲

CODE_GENDER = M (increased risk, impact 0.113)

▼

bureau_max_overdue_amt = 0.0 (decreased risk, impact 0.090)

Global Feature Importance

How each feature affects predictions across the whole applicant population.

Bridging ML output and credit policy

- Bins top-25 model features and surfaces any bin where default rate is materially above baseline
- Every rule backed by measured lift + support (n), not just model-internal importance
- Auditable by a policy team independent of trusting the ML model
- Example: Low-skill Laborers default at 2.12x baseline (n=2,093)

«

◆ Credit Risk Platform

AI-powered credit risk intelligence — NeoStats assignment

Overview

Data Exploration (EDA)

Risk Prediction

Explainability

Business Rules

Talk to Data

Deploy

:

Derived Business Rules

Plain-English underwriting rules mined from the model's learned patterns, each backed by measured lift on real validation data — not just model-internal importance.

If OCCUPATION_TYPE == 'Low-skill Laborers'
→ default rate 17.2% vs 8.1% baseline 2.12x lift (n=2,093)

If ORGANIZATION_TYPE == 'Transport: type 3'
→ default rate 15.8% vs 8.1% baseline 1.95x lift (n=1,187)

If EXT_SOURCE_3 in [-0.00, 0.37]
→ default rate 15.1% vs 8.1% baseline 1.87x lift (n=61,993)

If EXT_SOURCE_2 in [-0.00, 0.39]
→ default rate 14.3% vs 8.1% baseline 1.77x lift (n=76,716)

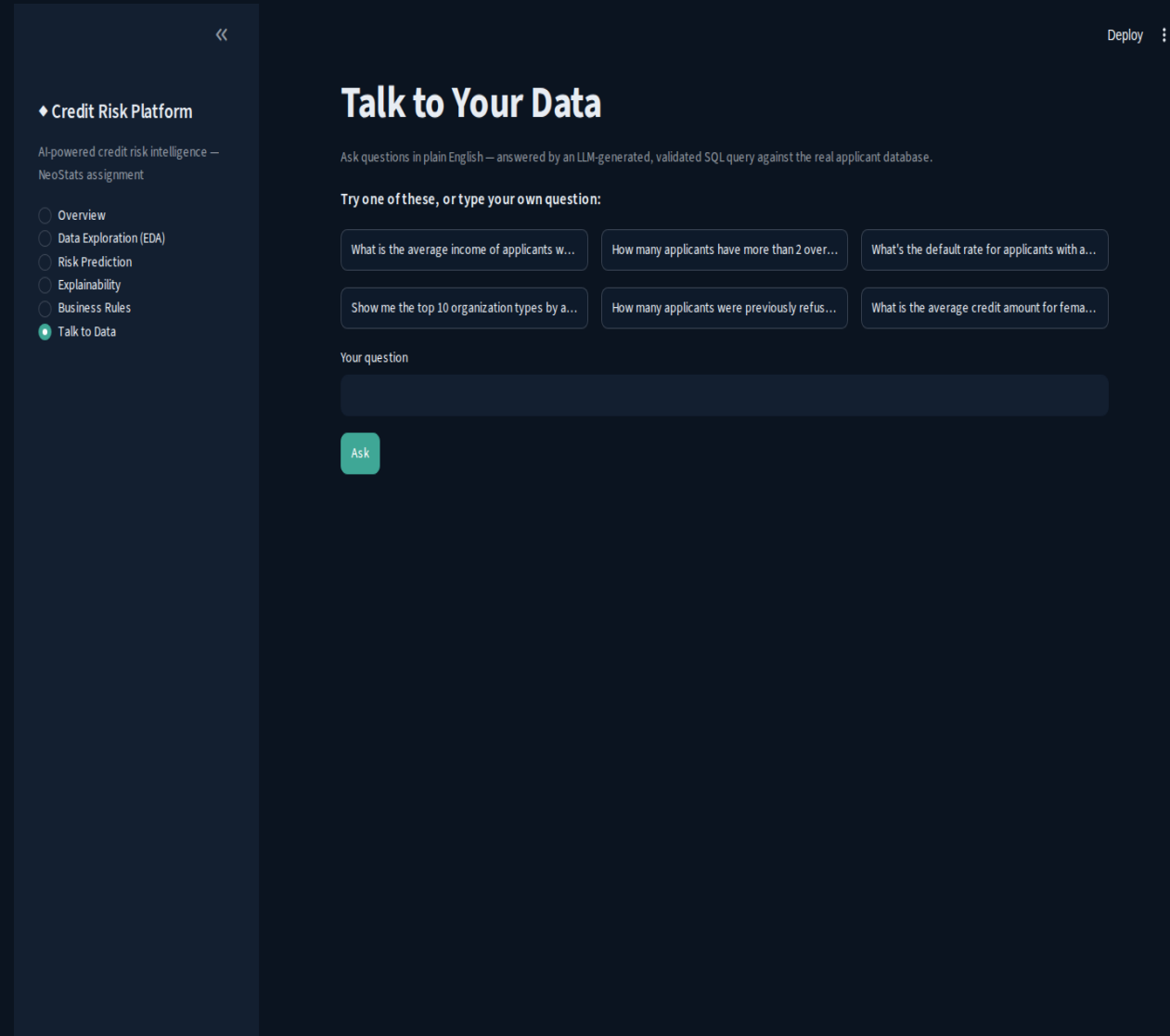
If EXT_SOURCE_1 in [0.01, 0.33]
→ default rate 13.6% vs 8.1% baseline 1.68x lift (n=33,534)

If ORGANIZATION_TYPE == 'Construction'
→ default rate 11.7% vs 8.1% baseline 1.45x lift (n=6,721)

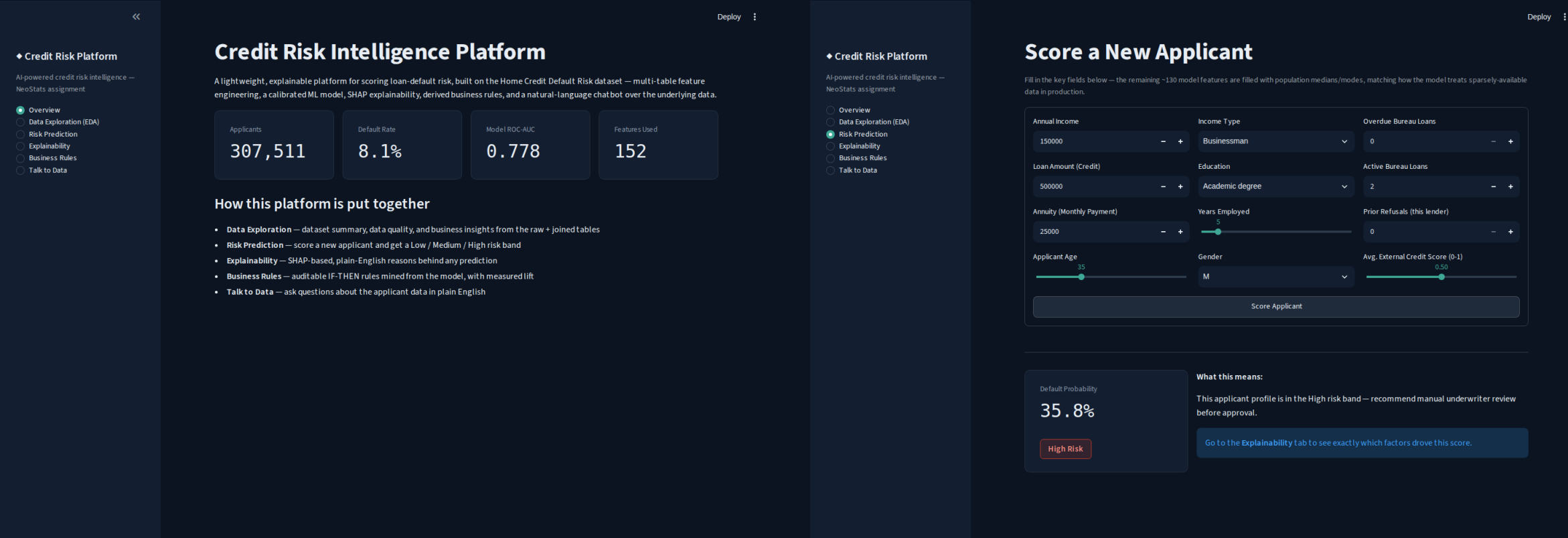
If ORGANIZATION_TYPE == 'Restaurant'
→ default rate 11.7% vs 8.1% baseline 1.45x lift (n=1,811)

NL-to-SQL with guardrails enforced in code

- Two-stage LLM calls: SQL generation (temp 0.1) → grounded answer summarization from real returned rows
- SQL validator blocks: DDL/DML keywords, chained statements, hallucinated tables/columns
- Auto row-limiting + read-only DB connection as a second defense layer
- Tested against real adversarial inputs (DROP TABLE, injection, fake tables) — all correctly blocked
- 6 verified working query patterns (exceeds the required 5)



One platform, five sections, real data throughout



Overview (KPIs from real data)

Risk Prediction (live scoring + risk band)

Streamlit • dark navy/teal design system • IBM Plex Sans • responsive KPI cards

One command to run the full platform

1

Clone & configure

git clone the repo, copy .env.example → .env, add a free Groq API key

2

Prep data & model

Download 5 Kaggle CSVs → run loader, preprocessor, train, evaluate, rules scripts

3

docker-compose up

Builds the image and launches Streamlit on port 8501 — single command

Dockerfile • docker-compose.yml • .env.example — all in the repo root, per the required structure

github.com/malavika-krishna24/credit-risk-platform

Known limitations & what's next

Two Kaggle tables not yet joined

bureau_balance.csv and credit_card_balance.csv would add further repayment signal on top of what's already used

Risk-band thresholds are defaults

Low/Medium/High cutoffs should be tuned against a real bank's approval/loss economics in production

Prediction form uses ~12 key fields

Remaining ~140 features filled from population medians/modes — a deliberate UX tradeoff for a demo interface

No monitoring/retraining pipeline

A production deployment needs drift detection and scheduled retraining — out of scope here

Thank you

Credit Risk Intelligence Platform — built end-to-end with real data, tested at every stage

github.com/malavika-krishna24/credit-risk-platform